Aniket Dombale_BSC23DS69

Task 1: Vague Prompt vs Well-Structured Prompt:-

Code:-

```
[ ]
!pip -q install groq
from groq import Groq
from getpass import getpass

client = Groq(api_key=getpass("Groq API key: ").strip())
model = "openai/gpt-oss-20b"

def ask(prompt):
    r = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
        max_tokens=200
    )
    return r.choices[0].message.content

def make_prompt(role, task, fmt, constraints):
    return f"Role: {role}\nTask: {task}\nFormat: {fmt}\nConstraints: {constraints}"

prompts = [
    "Explain machine learning.",
    make_prompt("Teacher", "Explain machine learning", "3 bullet points",
                "Use simple words for beginners."),
    make_prompt("HR assistant", "Write a polite leave email",
                "Email with subject", "Under 100 words."),
    make_prompt("Motivational speaker", "Encourage students before exams",
                "Short speech", "Use simple and positive language.")
]

for p in prompts:
    print("\nPROMPT:", p, "\nANSWER:", ask(p))
```

Output:-

```
Groq API key: ·····

PROMPT: Explain machine learning.
ANSWER: **Machine learning (ML)** is a subfield of artificial intelligence that focuses on building systems that can learn patterns from data, improve their performance over time, and make decisions or predictions without being explicitly programmed for each specific task.

---

## 1. Core Idea
Instead of writing a set of rules for every possible situation, ML models learn a mapping from input data to output labels (or actions) by observing many examples. The learning process adjusts internal parameters so that the model's predictions get closer to the true values.

---

## 2. Key Components

|

PROMPT: Role: Teacher
Task: Explain machine learning
Format: 3 bullet points
Constraints: Use simple words for beginners.
ANSWER: - **Learning from data:** Machine learning is like teaching a computer by giving it many examples, so it can spot patterns and make predictions on new, unseen data.
- **No explicit instructions:** Instead of writing step-by-step rules, we let the computer figure out the best way to solve a problem by itself.
- **Improves over time:** As the computer sees more data, it refines its guesses, getting better and more accurate with practice.

PROMPT: Role: HR assistant
Task: Write a polite leave email
Format: Email with subject
Constraints: Under 100 words.
ANSWER:

PROMPT: Role: Motivational speaker
Task: Encourage students before exams
Format: Short speech
Constraints: Use simple and positive language.
ANSWER: Good morning, everyone!

I know you're feeling a little nervous about the exams coming up. That's completely normal. But remember, you have worked hard and you are ready. Every minute you studied, every practice test you did, has built your confidence.

Take a deep breath. Think of your favorite song, your favorite place, or a moment that made you smile. Let that calm energy fill you. When you
```

Aniket Dombale_BSC23DS69

Task 2: Zero-Shot Text Classification:-

Code:-

```
!pip -q install transformers torch pandas
from transformers import pipeline
import pandas as pd

clf = pipeline("zero-shot-classification",
               model="facebook/bart-large-mnli")

data = [
    ("AI helps doctors detect diseases.", ["technology","sports","politics","food"]),
    ("India won a cricket match.", ["technology","sports","politics","food"]),
    ("I am excited about my new phone.", ["happy","sad","angry","worried"]),
    ("I feel disappointed today.", ["happy","sad","angry","worried"]),
    ("My bag has two zippers.", ["technology","sports","politics","food"])
]

rows = []
for sentence, labels in data:
    r = clf(sentence, candidate_labels=labels)
    print("\n", sentence)
    for l, s in zip(r["labels"], r["scores"]):
        print(l, round(s, 3))
    rows.append([sentence, r["labels"][0], round(r["scores"][0], 3)])

print(pd.DataFrame(rows, columns=["Sentence", "Top Label", "Score"]))
```

Output:-

```
Loading weights: 100% 515/515 [00:00<00:00, 561.29it/s]

 AI helps doctors detect diseases.
technology 0.996
food 0.002
sports 0.001
politics 0.001

 India won a cricket match.
sports 0.993
technology 0.004
food 0.002
politics 0.001

 I am excited about my new phone.
happy 0.994
worried 0.003
angry 0.002
sad 0.001

 I feel disappointed today.
sad 0.712
worried 0.183
angry 0.103
happy 0.002

 My bag has two zippers.
technology 0.82
sports 0.089
politics 0.048
food 0.043
                        Sentence   Top Label  Score
0  AI helps doctors detect diseases.  technology  0.996
1         India won a cricket match.      sports  0.993
2   I am excited about my new phone.       happy  0.994
3         I feel disappointed today.         sad  0.712
4            My bag has two zippers.  technology  0.820
```

Task 3: Chain-of-Thought Prompting:-

Code:-

```
problems = [
    ("A ₹1500 item has 20% discount and then 5% tax. Final price?", "1260"),
    ("3 workers make 48 items in 4 hours. How many do 5 workers make in 6 hours?", "120"),
    ("A bus travels 180 km at 60 km/h and 120 km at 40 km/h. Average speed?", "50")
]

rows = []
for q, correct in problems:
    direct = ask(q)
    steps = ask(q + " Explain your solution step by step.")
    print("\nProblem:", q, "\nDirect:", direct, "\nStep-by-step:", steps)
    rows.append([q, correct, direct, steps])

print(pd.DataFrame(rows, columns=[
    "Problem", "Correct Answer", "Direct Answer", "Step-by-Step Answer"
]))
```

Output:-

```
Problem: A ₹1500 item has 20% discount and then 5% tax. Final price?
Direct: First apply the 20 % discount:

- Discount = 20 % of ₹1,500 = 0.20 × 1,500 = **₹300**
- Price after discount = ₹1,500 − ₹300 = **₹1,200**

Now add the 5 % tax on the discounted price:

- Tax = 5 % of ₹1,200
Step-by-step: **Step-by-step calculation**

| Step | What we do | Formula | Result |
|------|------------|---------|--------|
| 1 | **Original price** of the item | – | ₹1500 |
| 2 | **Apply 20 % discount** | Discount = 20 % of ₹1500 = \(0.20 \times 1500\) | ₹300 |
| 3 | **Price after discount** | New

Problem: 3 workers make 48 items in 4 hours. How many do 5 workers make in 6 hours?
Direct: First find the production rate per worker per hour.

- 3 workers produce 48 items in 4 hours
  \[
  \text{Rate per worker-hour} = \frac{48\text{ items}}{3\text{ workers}\times 4\text{ hours}}
  = \frac{48}{12}=4\text{ items per worker-hour}.
  \]

Now use this rate for 5 workers working 6 hours:

\[
\
Step-by-step: **Step-by-step solution**

1. **Determine the total worker-hours in the first scenario**

   * 3 workers × 4 hours = **12 worker-hours**.

2. **Find

Problem: A bus travels 180 km at 60 km/h and 120 km at 40 km/h. Average speed?
Direct: **Average speed = 50 km/h**

**Why?**

1. **Total distance**
   \(180 \text{ km} + 120 \text{ km} = 300 \text{ km}\)

2. **Total time**
   * First leg: \( \frac{180\text{ km}}{60\text{ km/h}} = 3 \text{ h}\)
   * Second leg: \( \frac{120\text{
Step-by-step: **Step-by-step solution**
```

```
1. **Identify the two parts of the trip**

   | Part | Distance (km) | Speed (km/h) |
   |------|---------------|--------------|
   | 1    | 180           | 60           |
   | 2    | 120           | 40           |

2. **Compute the time spent in each part**


                                          Problem Correct Answer  \
0  A ₹1500 item has 20% discount and then 5% tax....           1260
1  3 workers make 48 items in 4 hours. How many d...            120
2  A bus travels 180 km at 60 km/h and 120 km at ...             50

                                       Direct Answer  \
0  First apply the 20 % discount:\n\n- Discount =...
1  First find the production rate per worker per ...
2  **Average speed = 50 km/h**\n\n**Why?**\n\n1. ...

                                  Step-by-Step Answer
0  **Step-by-step calculation**\n\n| Step | What ...
1  **Step-by-step solution**\n\n1. **Determine th...
2  **Step-by-step solution**\n\n1. **Identify the...
```

Aniket Dombale_BSC23DS69

Task 4: Image Captioning and Zero-Shot Image Classification:-

Code:-

```
!pip -q install transformers torch pillow pandas
from transformers import BlipProcessor, BlipForConditionalGeneration, pipeline
from google.colab import files
from PIL import Image
import pandas as pd

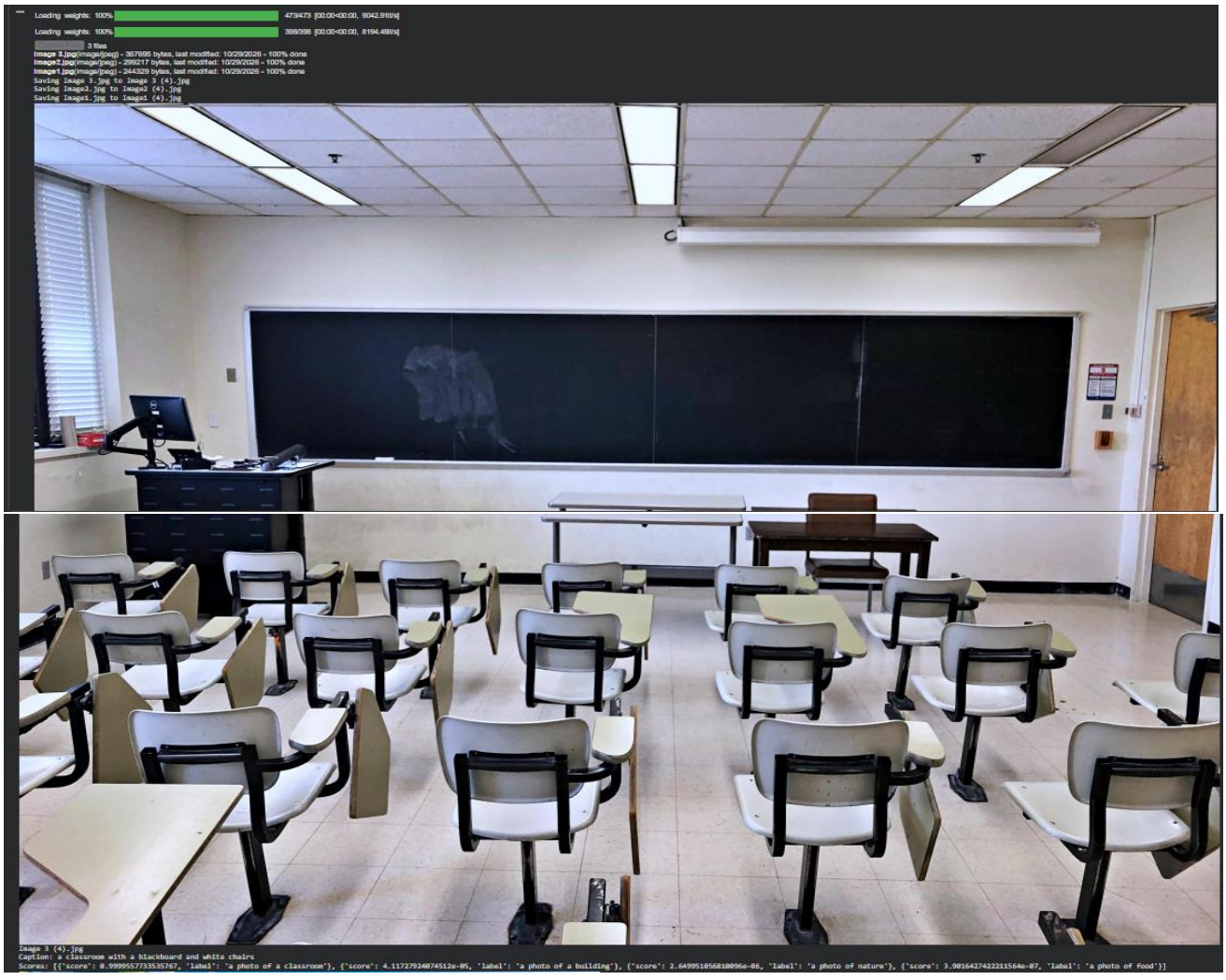
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model_img = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")
clip = pipeline("zero-shot-image-classification", model="openai/clip-vit-base-patch32")

labels = ["a photo of food", "a photo of nature",
          "a photo of a classroom", "a photo of a building"]
rows = []

for name in files.upload():
    img = Image.open(name).convert("RGB")
    display(img)
    inputs = processor(images=img, return_tensors="pt")
    caption = processor.decode(
        model_img.generate(**inputs, max_new_tokens=30)[0],
        skip_special_tokens=True
    )
    scores = clip(img, candidate_labels=labels)
    print(name, "\nCaption:", caption, "\nScores:", scores)
    rows.append([name, caption, scores[0]["label"], round(scores[0]["score"], 3)])

print(pd.DataFrame(rows, columns=["Image", "Caption", "Top Label", "Score"]))
```

Output:-





```
Image2 (4).jpg
Caption: the socta river in the mountains of socta valley, socta valley, socta valley, soc
Scores: [{'score': 0.9008425807952881, 'label': 'a photo of nature'}, {'score': 0.0065014458084067, 'label': 'a photo of a classroom'}, {'score': 0.0016040324609133635, 'label': 'a photo of a building'}, {'score': 0.0010519562056288123, 'label': 'a photo of food'}]
```



```
Image1 (4).jpg
Caption: a plate of food with rice, vegetables and sauces
Scores: [{'score': 0.9993778718574524, 'label': 'a photo of food'}, {'score': 0.0003087635559495538, 'label': 'a photo of a classroom'}, {'score': 0.0002518613182473938, 'label': 'a photo of nature'}, {'score': 6.234408647287637e-05, 'label': 'a photo of a building'}]
```

	Image	Caption
0	Image 3 (4).jpg	a classroom with a blackboard and white chairs
1	Image2 (4).jpg	the socta river in the mountains of socta vall...
2	Image1 (4).jpg	a plate of food with rice, vegetables and sauces

	Top Label	Score
0	a photo of a classroom	1.000
1	a photo of nature	0.991
2	a photo of food	0.999

Task 5: AI-Assisted Data Cleaning:-

Code:-

```
!pip -q install groq pandas
import pandas as pd
from groq import Groq
from getpass import getpass

client = Groq(api_key=getpass("Groq API key:"))

df = pd.DataFrame({
    "Name": ["ANIKET", "arya", "Dip", "NEHA", None, "sam", "Pooja", "JAY"],
    "Marks": [85, 92, None, 110, 76, "absent", 88, 65]
})

print("Before Cleaning:\n", df)

prompt = """Write pandas code only to clean df:
standardize names, convert marks to numeric,
replace invalid marks outside 0-100 with NaN,
and keep missing values as NaN."""

code = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[{"role": "user", "content": prompt}]
).choices[0].message.content

print("\nAI Generated Code:\n", code)

# Reviewed, safe cleaning code
clean = df.copy()
clean["Name"] = clean["Name"].str.strip().str.title()
clean["Marks"] = pd.to_numeric(clean["Marks"], errors="coerce")
clean.loc[~clean["Marks"].between(0, 100), "Marks"] = float("nan")

print("\nAfter Cleaning:\n", clean)
```

Output:-

```
Groq API key:··········
Before Cleaning:
      Name   Marks
0  ANIKET      85
1    arya      92
2     Dip    None
3    NEHA     110
4    None      76
5     sam  absent
6   Pooja      88
7     JAY      65

AI Generated Code:
 ```python
import numpy as np

# Standardise name column (strip whitespace, title-case)
df['name'] = df['name'].str.strip().str.title()

# Convert marks to numeric, coercing invalid values to NaN
df['marks'] = pd.to_numeric(df['marks'], errors='coerce')

# Replace marks outside 0–100 with NaN (keeps existing NaNs untouched)
df['marks'] = df['marks'].where((df['marks'] >= 0) & (df['marks'] <= 100), np.nan)
```


After Cleaning:
      Name Marks
0 Aniket   85.0
1   Arya   92.0
2   Dip   NaN
3   Neha   NaN
4   None   76.0
5   Sam   NaN
6   Pooja   88.0
7   Jay   65.0
```

Aniket Dombale_BSC23DS69

Task 6: Jinja2 Templating for a Flask Page:-

Code:-

```
!pip -q install jinja2
from jinja2 import Template
from IPython.display import HTML, display

def get_ai_response(prompt):
    return "AI answer: " + prompt

questions = ["What is AI?", "What is ML?", "What is NLP?", "What is GenAI?"]
chat = [{"q": q, "a": get_ai_response(q)} for q in questions]

template = Template("""
<html><body>
<h2>Hello, {{ name }}!</h2>
<h3>AI Response</h3><p>{{ answer }}</p>
<h3>Chat History</h3>
{% for item in chat %}
<p><b>Q:</b> {{ item.q }}<br><b>A:</b>
{{ item.a if item.a else "No answer yet" }}</p>
{% endfor %}
</body></html>
""")

page = template.render(name="Aniket", answer=get_ai_response("Explain GenAI"), chat=chat)

print(page)
display(HTML(page))
with open("ai_page.html", "w") as f:
    f.write(page)

print("HTML page saved!")
```

Output:-

```
<html><body>
<h2>Hello, Aniket!</h2>
<h3>AI Response</h3><p>AI answer: Explain GenAI</p>
<h3>Chat History</h3>

<p><b>Q:</b> What is AI?<br><b>A:</b>
AI answer: What is AI?</p>

<p><b>Q:</b> What is ML?<br><b>A:</b>
AI answer: What is ML?</p>

<p><b>Q:</b> What is NLP?<br><b>A:</b>
AI answer: What is NLP?</p>

<p><b>Q:</b> What is GenAI?<br><b>A:</b>
AI answer: What is GenAI?</p>

</body></html>
```

Hello, Aniket!

AI Response

AI answer: Explain GenAI

Chat History

Q: What is AI?
A: AI answer: What is AI?

Q: What is ML?
A: AI answer: What is ML?

Q: What is NLP?
A: AI answer: What is NLP?

Q: What is GenAI?
A: AI answer: What is GenAI?

HTML page saved!